

AMM 去中心化交易所仿真系统技术报告

基于恒定乘积模型的 DeFi 核心机制模拟、系统架构与实验验证

项目	说明
系统类型	可交互 Web 仿真系统
技术栈	Python、Flask、JavaScript、ECharts、JSON
核心机制	AMM、Swap、流动性池、LP Token、手续费、无常损失
报告用途	课程项目技术报告，正文超过 3000 字

一、问题定义：模拟的 DeFi 机制与金融问题

本项目模拟的核心 DeFi 机制是自动做市商 Automated Market Maker, 简称 AMM。AMM 是去中心化交易所中最具有代表性的机制之一, 典型代表包括 Uniswap V2 一类基于恒定乘积模型的交易协议。与传统中心化交易所不同, AMM 不依赖订单簿撮合买卖双方, 而是通过一个包含两种资产的流动性池完成交易。交易者输入一种资产并从池中取出另一种资产, 价格由池中资产储备比例自动决定。

传统金融市场和中心化交易所撮合交易时通常依赖订单簿。订单簿要求买方和卖方同时存在, 并且需要有足够密集的挂单才能形成稳定价格。对于长尾资产、早期资产或流动性较弱的市场, 订单簿容易出现买卖价差大、深度不足、成交困难等问题。AMM 的金融意义在于, 它把市场流动性从被动挂单转化为可被任何人存入的资金池, 使用户可以随时与池子成交, 从而降低交易匹配成本, 提高资产交易的连续性。

本项目选取 ETH/USDC 交易对作为仿真对象。系统初始化时, 池中包含 100 ETH 和 200000 USDC, 因此初始价格为 $1 \text{ ETH} = 2000 \text{ USDC}$ 。系统围绕几个核心问题展开: 第一, 没有订单簿时交易价格如何形成; 第二, 用户执行 swap 后池子储备如何变化; 第三, 大额交易为什么会更高滑点和价格冲击; 第四, 流动性提供者如何存入资产并获得 LP Token; 第五, LP 为什么可能面对无常损失; 第六, 手续费如何累计并体现为协议收益或 LP 收益来源。

从课程项目角度看, 本系统不是追求完整 DeFi 生态, 而是聚焦 AMM 这一核心机制的最小可行产品。项目实现了可交互 Web 界面、参数配置、预设事件场景、结果图表与交易日志, 能够支持用户从操作层面观察 DeFi 协议内部状态变化。因此, 它既满足 DeFi 机制模拟的核心要求, 也便于通过演示说明数学模型与实际行为之间的关系。

二、数学模型：恒定乘积、交易报价、LP 与无常损失

2.1 恒定乘积模型

本项目采用 AMM 中最经典的恒定乘积公式。设池中 Token X 的储备为 x , Token Y 的储备为 y , 则池子状态满足 $x * y = k$, 其中 k 为恒定乘积。在本项目中 Token X 为 ETH, Token Y 为 USDC。池内价格可以用储备比例表示, 即 $P = y / x$, 含义是 1 个 ETH 对应多少 USDC。当 $x = 100, y = 200000$ 时, $P = 200000 / 100 = 2000$ 。

$$x * y = k$$

$$P = y / x$$

该模型的关键思想是, 用户交易会改变 x 和 y 的值, 但交易后的新储备仍然位于同一条恒定乘积曲线上。若用户输入 ETH, 则池中 ETH 增加, USDC 减少; 由于乘积保持不变, ETH 储备增加会使 USDC 储备以非线性方式下降。因此, 交易越大, 用户实际成交价格越偏离交易前的边际价格, 这就是 AMM 滑点的来源。

2.2 Swap 报价公式

设用户输入 Δx 个 ETH, 手续费率为 f 。本项目默认手续费率为 0.003, 即 0.3%。扣除手续费后实际进入曲线计算的数量为 $\Delta x_{\text{eff}} = \Delta x * (1 - f)$ 。交易后 ETH 储备变为 $x' = x + \Delta x_{\text{eff}}$ 。为了保持恒定乘积, 新的 USDC 储备为 $y' = k / x'$ 。用户得到的 USDC 数量为 $\Delta y = y - y'$ 。

$$\text{fee} = \Delta x * f$$

$$\Delta x_{\text{eff}} = \Delta x * (1 - f)$$

$$x' = x + \Delta x_{\text{eff}}$$

$$y' = k / x'$$

$$\text{输出 USDC} = y - y'$$

当用户输入 USDC 兑换 ETH 时，公式对称变化。设用户输入 Δy 个 USDC，则 $\Delta y_{\text{eff}} = \Delta y * (1 - f)$ ，新的 USDC 储备为 $y' = y + \Delta y_{\text{eff}}$ ，新的 ETH 储备为 $x' = k / y'$ ，用户获得的 ETH 为 $x - x'$ 。由于模型是非线性的，同样价值的大额交易会小额交易带来更明显的价格影响。

2.3 价格冲击与手续费

项目中将价格冲击定义为交易前后池内价格变化幅度： $\text{price_impact} = |P_{\text{after}} - P_{\text{before}}| / P_{\text{before}}$ 。采用绝对值可以避免不同交易方向导致正负号混淆，更符合前端展示中“滑点幅度”的语义。手续费由 FeeManager 模块统一计算和累计，避免重复统计。交易手续费可以理解为协议或 LP 收益的来源，系统在数据日志页展示累计手续费和交易量。

$$\begin{aligned} P_{\text{before}} &= y / x \\ P_{\text{after}} &= y' / x' \\ \text{price_impact} &= |P_{\text{after}} - P_{\text{before}}| / P_{\text{before}} \end{aligned}$$

2.4 LP Token 与无常损失

流动性提供者需要按照当前池子比例同时存入两种资产。初始 LP Token 数量使用 $\text{sqrt}(x * y)$ 计算，后续添加流动性时，新增 LP Token 与存入资产占当前池子的比例相关。例如用户存入 Δx 个 ETH，则 $\text{LP_mint} = \Delta x / x * \text{LP_total}$ ，同时要求 $\Delta y / \Delta x$ 与 y / x 接近。如果比例偏差过大，系统拒绝添加流动性，避免破坏池子价格。

$$\begin{aligned} \text{LP_initial} &= \text{sqrt}(x * y) \\ \text{LP_mint} &= \Delta x / x * \text{LP_total} \\ \text{添加条件: } \Delta y / \Delta x &\approx y / x \end{aligned}$$

LP 的风险主要体现在无常损失。无常损失衡量的是做 LP 相比单纯持有原资产的价值差。设价格变化倍数为 $r = \text{当前价格} / \text{初始价格}$ ，则经典无常损失公式为 $\text{IL} = 2 * \text{sqrt}(r) / (1 + r) - 1$ 。当 r 偏离 1 越远，无常损失越大。项目中的 PositionManager 模块计算 LP 当前价值、HODL 价值、价格变化倍数和无常损失百分比，并在前端的盈亏分析页展示。

$$\begin{aligned} r &= P_{\text{current}} / P_{\text{initial}} \\ \text{IL} &= 2 * \text{sqrt}(r) / (1 + r) - 1 \end{aligned}$$

三、系统架构：模块划分、事件流与数据结构

系统采用前后端分离思想，但部署上保持轻量化。后端使用 Flask 提供 REST API，前端使用 HTML、CSS、JavaScript 和 ECharts 进行交互与可视化。系统的整体结构可以分为五层：前端交互层、API 层、仿真控制层、核心业务层、数据配置与日志层。这样的划分使数学模型、场景控制和可视化展示彼此解耦，便于调试和演示。

层级/模块	主要文件	职责
前端交互层	templates/index.html、static/js/app.js、static/css/style.css	提供仪表盘、代币交换、流动性池、盈亏分析、仿真控制、数据日志等页面。
API 层	app.py	定义 Flask 路由，接收前端请求，组织核心模块返回 JSON 数据。
核心池模型	core/liquidity_pool.py	实现恒定乘积公式、报价、执行交易、添加/移除流动性和池状态查询。

层级/模块	主要文件	职责
交易与费用	core/swap_engine.py、core/fee_manager.py	协调交易执行、手续费计算、手续费累计和结果返回。
仓位与损益	core/position_manager.py	管理 LP 仓位，计算 LP 价值、HODL 价值和无常损失。
仿真控制	core/simulation.py、core/oracle_simulator.py	加载场景、按时间步执行事件，生成固定种子的预言机价格序列。
数据层	data/*.json、core/data_logger.py	保存池配置、用户、场景、预言机数据和交易日志。

典型 swap 事件流如下：用户在前端输入交易数量，前端调用 /api/swap/quote 请求报价；后端 SwapEngine 调用 LiquidityPool 计算输出数量、手续费和价格影响；用户确认后，前端调用 /api/swap/execute；SwapEngine 执行交易，LiquidityPool 更新储备，FeeManager 累计手续费，DataLogger 记录交易；最后前端刷新仪表盘、价格图、交易日志和用户余额。

前端输入交易 → /api/swap/quote → LiquidityPool 计算报价
用户确认交易 → /api/swap/execute → SwapEngine 执行交易
更新储备 → 累计手续费 → 写入日志 → 前端刷新图表

主要数据结构包括池状态、用户余额、场景事件和交易日志。池状态包含 token_x、token_y、reserve_x、reserve_y、k、fee_rate、total_lp_tokens、current_price 和 price_history。用户余额结构包含用户编号、名称、类型、ETH、USDC 与 lp_tokens。场景事件结构包含 step、user_id、action、token_in、amount 或 token_x_amount、token_y_amount、lp_tokens 等字段。交易日志记录交易编号、用户、输入输出资产、手续费、价格影响和时间戳。

四、仿真设计：时间推进、用户行为与随机性控制

本项目采用离散时间步仿真，而不是连续时间仿真。场景文件 scenarios.json 中的每个事件都绑定一个 step。SimulationController 从 step = 0 开始推进，每执行一步，就筛选所有 step 等于当前步数的事件并执行。执行结束后，系统记录该步的池状态、AMM 价格、预言机价格和事件结果。这样设计的优点是逻辑清晰、结果可复现、适合课堂演示，也方便对不同场景进行对比。

用户行为模型在 users.json 中定义。系统包含普通交易者、流动性提供者、套利交易者、研究用户和大额交易者。普通交易者主要用于模拟常规 swap；流动性提供者用于演示添加和移除流动性；套利交易者用于高频交易或价格偏差场景；研究用户用于小额教学操作；大额交易者用于观察价格冲击。虽然这些用户不是复杂智能体，但已经足够支撑 AMM MVP 的核心实验。

用户类型	代表用户	建模目的
普通交易者	Alice	执行中小额 swap，观察常规交易下价格与储备变化。
流动性提供者	Bob	添加和移除流动性，验证 LP Token 与仓位分析。
套利者	Charlie	进行较高频交易，模拟市场偏差下的快速交易行为。
研究用户	Diana	用于教学实验和无常损失分析。
大额交易者	Eve	执行大额 swap，观察极端价格冲击和滑点。

随机性控制由 OracleSimulator 完成。预言机模拟器使用固定随机种子 seed = 42，并采用对数正态随机游走生成外部市场价格。价格更新形式可以表示为 $new_price = last_price * \exp(\log_return)$ ，

其中 `log_return` 来自均值为 0、波动率为 `volatility` 的高分布。固定种子的意义在于每次运行时价格序列相同，便于调试、复现实验和现场演示。

系统目前提供四类预设场景：基础交易场景、大额冲击场景、流动性提供者场景和套利场景。基础交易场景展示多用户在不同时间点进行 ETH/USDC 交换；大额冲击场景强调交易规模对价格冲击的影响；流动性提供者场景展示 LP 进入和退出池子的过程；套利场景通过高频交易模拟价格偏差下的交易行为。四类场景共同覆盖 AMM 的主要机制，符合聚焦 MVP 的项目原则。

五、实验与验证

5.1 基础功能测试

基础测试首先验证初始化状态。系统读取 `pool_config.json` 后建立 ETH/USDC 池，初始储备为 100 ETH 与 200000 USDC，初始价格为 2000 USDC/ETH，恒定乘积 $k = 20000000$ 。该状态能够被 `/api/pool/state` 正确返回，并在仪表盘中展示。随后测试 `swap` 报价接口，输入 1 ETH 时，系统返回输出 USDC、手续费、交易前价格、交易后价格和价格影响。该结果说明报价逻辑能够使用手续费后的有效输入计算曲线变化。

第二项基础验证是恒定乘积守恒。执行交易后，系统更新 `reserve_x` 与 `reserve_y`，并重新计算 k 。由于手续费没有作为有效输入参与恒定乘积曲线计算，交易后的 $x' * y'$ 仍然保持在原始 k 附近。项目使用 `Decimal` 提高计算精度，减少浮点误差对验证结果的影响。第三项基础验证是流动性添加比例检查。用户添加流动性时必须匹配当前池子比例，如果使用错误比例，系统会返回“流动性比例不匹配”错误。

5.2 极端场景测试

AMM 项目的极端场景不应写成借贷协议中的价格暴跌清算，而应围绕交易冲击、流动性不足和错误流动性比例展开。大额冲击场景中，Eve 连续执行大额 ETH 或 USDC 交易，系统可以观察到价格明显偏移，滑点高于小额交易。这验证了恒定乘积曲线的非线性特征：交易规模越接近池子深度，对储备比例的改变越大，价格冲击越明显。

流动性不足测试可以通过输入接近池子储备的大额交易触发。`LiquidityPool` 在报价阶段会判断有效输入是否过大，如果超出可接受范围，则抛出 `InsufficientLiquidityError`，避免产生不合理输出。比例错误测试则验证 LP 添加流动性的约束。当池子已经经过多次 `swap` 后，当前价格不再等于初始价格，如果仍按初始 $1 \text{ ETH} = 2000 \text{ USDC}$ 添加流动性，系统会拒绝。这一测试体现了 AMM 中 LP 必须按当前池价提供等比例资产的原则。

5.3 对比分析

项目支持多个维度的对比分析。第一是小额交易与大额交易对比。小额交易对池子储备比例影响较小，滑点较低；大额交易会明显改变 x 与 y 的比例，价格影响更高。第二是添加流动性前后的对比。池子深度增加后，同样规模的交易占池子比例下降，因此价格冲击减弱。第三是 LP 与 HODL 的对比。`PositionManager` 计算 LP 当前份额价值和如果单纯持有原资产的 HODL 价值，从而展示无常损失。

测试项目	验证目标	结果说明
初始化测试	检查储备、价格、 k 是否正确	100 ETH、200000 USDC、价格 2000、 $k=20000000$ 。
Swap 报价测试	检查输出数量、手续费、滑点	接口能返回完整 <code>quote</code> ，价格影响为正的变化幅度。

测试项目	验证目标	结果说明
场景完整性测试	四个预设场景是否顺利跑完	基础交易、大额冲击、LP 场景、套利场景均可运行完成。
比例错误测试	错误比例添加流动性是否被拒绝	系统拒绝不匹配比例，保护池子价格逻辑。
手续费测试	手续费是否重复累计	修复后 FeeManager 与池状态累计手续费一致。

在验证过程中，项目曾暴露出几个真实问题：USDC 换 ETH 时 `price_impact` 因交易方向不同出现负数；部分场景中的流动性添加参数仍按初始价格写死，导致场景事件失败；手续费曾在 `LiquidityPool` 与 `FeeManager` 中重复累计。通过修复后，价格影响改为绝对变化幅度，场景参数根据交易后的池子比例调整，手续费由 `FeeManager` 统一累计。修复后的系统更适合正式演示和实验报告展示。

六、团队分工与反思

团队分工按照模块职责进行描述。

成员 A 主要负责后端核心模型，实现 `LiquidityPool`、恒定乘积公式、swap 计算、添加与移除流动性。成员 B 主要负责仿真控制与数据设计，完成用户配置、场景配置、事件步进和用户余额更新。成员 C 主要负责前端交互与可视化，实现仪表盘、代币交换页、流动性池页、盈亏分析页和 ECharts 图表。

项目难点主要集中在模型与工程实现的边界上。

第一，AMM 的数学公式看似简单，但交易方向不同会影响价格变化方向，如果直接使用带符号的价格变化，前端滑点展示可能产生误解。第二，流动性添加不是固定比例，而是必须根据当前池子状态动态计算，因此预设场景中的参数不能简单沿用初始价格。第三，手续费既属于交易逻辑，又属于收益统计，如果模块职责不清晰，就容易重复累计。第四，前端图表展示需要与后端 JSON 字段保持一致，否则会出现数据存在但展示异常的问题。

后续改进方向包括：支持用户在前端自定义手续费率、初始池子储备和交易规模；加入更真实的套利机器人策略，使机器人根据池内价格与预言机价格差自动交易；支持更多交易对和多池仿真；增加单元测试框架，自动验证恒定乘积、手续费、LP Token 和无常损失；引入真实历史价格数据，让仿真场景更贴近市场；增加导出实验结果功能，方便生成课程报告。若进一步扩展，还可以加入 Uniswap V3 的集中流动性机制，或加入借贷协议中的抵押率与清算机制，但这些属于下一阶段，不应影响当前 MVP 对 AMM 核心机制的聚焦。

总体来看，本项目围绕 DeFi 中最基础也最典型的 AMM 机制，完成了从数学模型、后端计算、前端交互到实验验证的完整闭环。系统没有追求完整生态，而是聚焦自动做市、价格冲击、流动性提供、手续费和无常损失这几个核心问题。通过可交互 Web 页面和预设仿真场景，用户能够直观看到交易如何改变池子储备、价格如何由储备比例决定、LP 如何承担风险并获得手续费收益。因此，该项目能够较好地支撑 DeFi 机制课程展示和技术报告要求。正文中文字符统计约 4323 字。